



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

观察者模式（Observer）

一、意图（Intent）

定义对象之间的一对多依赖关系，使一个对象状态变化时，所有相关方都能收到通知并自行处理。

站在嵌入式 C 的角度，这个模式控制的是：

- 谁发布状态变化
- 谁订阅这类变化
- 通知在什么上下文里执行
- 发布者是否需要知道具体接收者

一句话记住它：

事件源只负责发通知，不负责点名谁来处理。

二、动机（Motivation）

在嵌入式系统里，很多状态变化都会牵动多个模块：

- 按键变化要通知 UI、功耗管理、业务逻辑
- 网络连上以后要通知应用、时间同步、云端上报
- 传感器数据就绪要通知控制回路、记录模块、调试输出
- 电源状态变化要通知显示、充电管理、限频策略

最原始的写法通常是：

- 状态源直接调用多个模块
- 谁要响应，就在状态源里加一个回调
- 模块越来越多时，状态源变成一个分发中心

这会带来几个很典型的问题：

1. 发布者耦合过多接收者

状态源本来只该关注“状态是否变化”，结果却知道：

- 哪些模块需要通知
- 通知顺序
- 某些模块是否允许阻塞
- 某些模块是不是只能在线程上下文处理

2. 新增需求会不断改动源模块

每多一个消费者，状态源就得加注册项、加回调、加 if 分支。

3. 时序和上下文容易混乱

在嵌入式里，通知是不能脱离执行上下文谈的：

- ISR 能不能直接通知？
- 回调是否允许阻塞？
- 是否要切到 work queue？
- 一个慢订阅者会不会拖垮整个发布链？

所以观察者模式在嵌入式里最重要的价值不是“有回调”，而是：

把状态发布与后续响应解耦，并明确通知机制的上下文规则。

三、适用性（Applicability）

适合使用观察者模式的条件通常有：

- 某个状态变化需要通知**多个模块**
- 发布者**不应该知道**所有接收者细节
- 接收者集合可能**增加或裁剪**
- 希望把业务逻辑做成**订阅式拼装**
- 需要明确同步通知、异步通知、ISR 通知等规则

常见场景：

- 事件总线
- 网络状态变化

- 传感器数据就绪通知
- 电源与系统模式切换通知
- GUI / 输入子系统消息广播

四、结构（Structure）

```
事件源 / Subject
|
+----> Observer A
+----> Observer B
+----> Observer C
```

在嵌入式 C 里更常见的写法是：

```
发布者
|
v
事件通道 / 通知中心 / 观察者列表
|
+-- 监听器回调
+-- 线程订阅者
+-- 异步工作队列处理器
```

也就是说，很多工程不会硬做成“一个对象 + 一组对象”，而是做成：

- 一个事件源
- 一个主题 / 通道 / bus
- 多个订阅者

五、参与者（Participants）

1. Subject（主题 / 发布者）

负责：

- 维护观察者关系
- 在状态变化时发布通知

在嵌入式里，它可能是：

- 网络管理器
- 按键模块
- 传感器采集模块
- 电源管理模块
- 总线事件通道

2. Observer（观察者 / 订阅者）

负责接收通知并作出响应。

在 C 里通常体现为：

- 回调函数
- 订阅线程
- work item 处理函数

3. ConcreteObserver（具体观察者）

具体实现各自处理逻辑。

例如：

- UI 更新
- 日志记录
- 控制策略切换
- 故障上报

六、协作（Collaborations）

整体过程通常是：

1. 观察者先注册到某个主题 / 通道
2. 主题状态变化或接收到事件
3. 主题向所有观察者发通知
4. 每个观察者在自己的处理逻辑里响应

这里最关键的不是“通知出去”本身，而是：

- 是同步还是异步
- 是直接在发布上下文执行还是延后执行
- 是否允许丢事件
- 是否有缓冲区和背压

所以在嵌入式里，观察者模式必须和**执行上下文**一起讲。

七、效果（Consequences）

收益

1. 发布者与处理者解耦

事件源不需要知道所有消费方，也不需要直接依赖它们。

2. 系统扩展更自然

新增一个观察者，通常只需注册，不必改事件源主逻辑。

3. 适合裁剪式产品线

某些订阅者开配置即可编译进来，不开则完全不参与。

4. 更适合做多模块拼装

输入、功耗、连接状态这类公共事件，很适合做成统一通知面。

代价

1. 调试链路会变长

事件是谁发的、谁收了、谁又触发了下一步动作，跟踪成本会上升。

2. 慢观察者可能拖慢系统

如果采用同步通知，某个处理者执行过慢会影响整个发布路径。

3. 需要严格定义上下文规则

ISR、线程、work queue、锁持有状态，都必须界定清楚。

八、实现要点（Implementation）

1. 明确通知上下文

在嵌入式里，第一件事不是定义事件结构体，而是先回答：

- ISR 能否发布？
- 观察者回调在哪执行？
- 是否允许阻塞？
- 是否允许再次发布事件？

2. 数据传递要考虑拷贝和所有权

常见方式有：

- 只发 event id
- 发轻量结构体副本
- 发共享内存引用
- 发队列消息

不同做法会影响：

- RAM 占用
- 生命周期管理
- 并发风险

3. 注册与注销必须安全

动态注册、静态注册、启动期注册，行为要一致。

尤其要处理：

- 遍历时注销
- 回调中再次注册
- 模块卸载后悬挂指针

4. 事件粒度不要失控

事件太细，系统会到处广播；

事件太粗，观察者又要大量二次判断。

九、典型应用

1. Zephyr zbus

Zephyr 的 zbus 明确提供 channel 与 observer 的组织方式，支持 many-to-many 通信，并区分 listener、async listener、subscriber 等观察者形态。发布和读取操作还说明了其可在 RTOS 线程上下文甚至 ISR 中使用，这正是嵌入式观察者模式最关键的“上下文规则”。

2. ESP-IDF 默认事件循环

ESP-IDF 的 `esp_event` 把系统事件投递到默认事件循环，组件和应用通过注册 handler 来接收 Wi-Fi、IP 等事件；默认事件循环的 handle 对用户隐藏，应用面对的是“注册处理者”，不是直接耦合到事件源。

3. 网络、功耗、输入子系统事件广播

这类模块几乎都天然是一源多收：

- 一个连接状态变化，多个模块都要感知
- 一个低电量事件，多个策略都要调整
- 一个按键动作，多个上层功能都可能响应

总结

观察者模式在嵌入式 C 里，就是把“状态变化后的多方响应”从发布者身上拆下来，做成可订阅、可裁剪、可控上下文的通知机制。